

AULA 8

Simuladores com ROS 2: Simulador Gazebo

CONTINUAÇÃO DA AULA 7: CONTROLE DO RRBOT NO ROS2_CONTROL

Exemplo vindo da documentação do ros2_control

Controle de um robô planar com 2 juntas revolutas

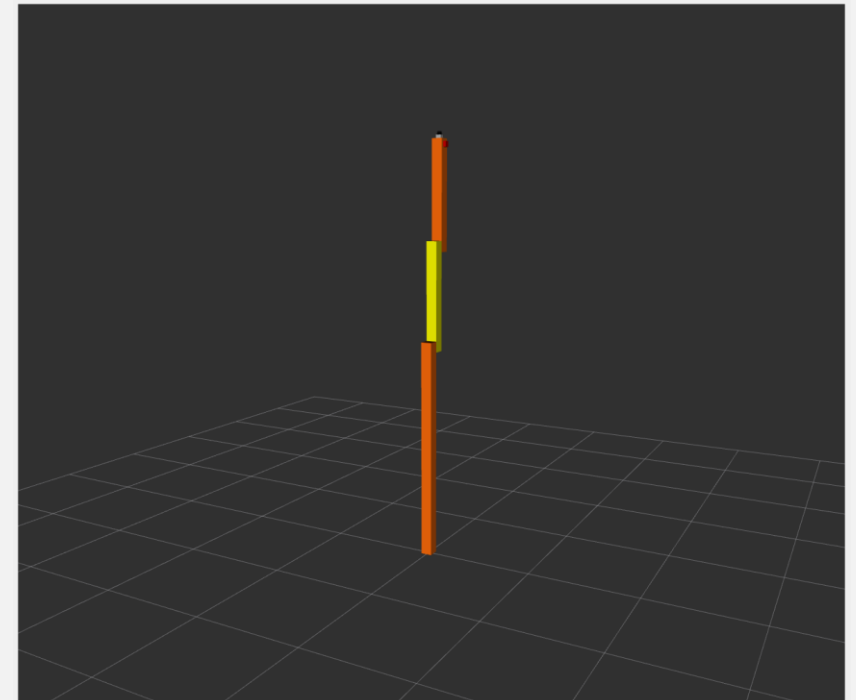
Plugin de hardware é implementado com só 1 interface

Instruções para acompanhar a demonstração:

https://control.ros.org/jazzy/doc/ros2_control_demos/example_1/doc/userdoc.html

É necessário ter [instalado](#) o repositório com os pacotes do ros2_control_demos

Exemplo 7 também é completo, para quem tiver interesse



Vista do rrbot no RViz2.

CONTINUAÇÃO DA AULA 7: EXEMPLO: INTERFACE COM MOTORES DYNAMIXEL VIA ROS2_CONTROL

Servomotores com interface em `ros2_control`, disponibilizada nativamente pela fabricante

- Controle pode ser de posição, velocidade ou corrente
- Pacote [dynamixel_hardware_interface](#) implementa uma `HardwareInterface`, que se comunica com a SDK para manda os comandos e ler os estados presentes no microcontrolador do motor
- Tags no urdf permitem customizar o *setup*; montagem, tipo de comunicação, ganhos internos dos motores, etc.

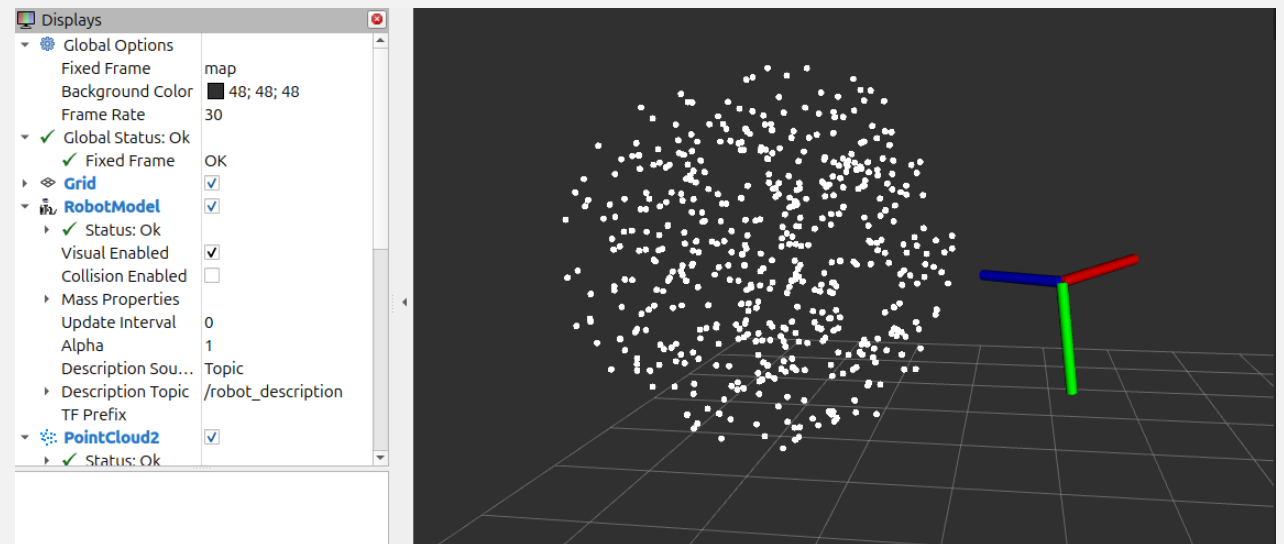


Vídeo exemplo do motor Dynamixel XH430-V350 com interface por ROS 2

(1/2) EXERCÍCIO DA "ÚLTIMA" AULA

Vasculhe a nuvem de pontos (abordagem livre, sejam criativos) e eliminem todos os pontos que não estejam em uma banda de até 0.5m da altura do frame **drone_frame** em relação à **map**.

Após isso publiquem a nuvem de pontos resultante em um tópico do tipo **sensor_msgs/msg/PointCloud2**.



SIMULADORES EM ROBÓTICA

Simuladores são essenciais no desenvolvimento de robôs mais complexos

- Robôs são peças de hardware (bem) caras
- Avaliação do desempenho do sistema em condições controladas
- Auxílio no desenvolvimento de estratégias de controle, verificar modificações no hardware
- Robôs industriais tipicamente têm um simulador proprietário para programação offline
 - Em alguns casos, robôs que usam estratégias de aprendizado são "treinados" em simulação antes da implementação no hardware (aprendizado por reforço, *sim2real*)

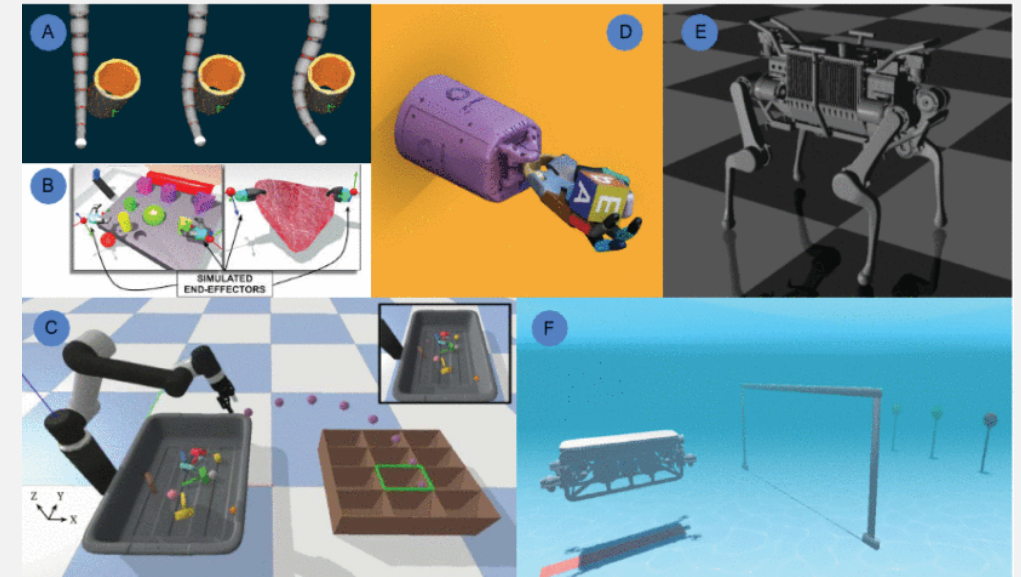


Simulação de linha de montagem com robôs Kuka em simulador da Siemens

O QUÊ É UM SIMULADOR EM ROBÓTICA?

O que faz um simulador?

- Integração com motores de física
 - Tipicamente apenas mecânica clássica, mas por vezes também motores especializados
- Modelos para descrição e simulação de cadeias cinemáticas, sensores e atuadores
- Interface para o programador do(s) robô(s)
 - Seja por APIs especializadas, seja por meios mais agnósticos ao robô (e.g. torque nas juntas)
- Interface gráfica (ou pelo menos uma forma de exportar dados resultantes da simulação)
- Capacidade de importar/carregar cenas, malhas, objetos, etc.



Algumas cenas em diferentes simuladores de robótica. (a) Soft robotics ([SOFA](#)); (b) robótica médica ([AMBE](#)); (c) Manipulação ([PyBullet](#)); (d) Manipulação fina ([MuJoCo](#)); (e) Plataforma com pernas ([RaiSim](#)); (f) Veículos subaquáticos ([URSim](#))

Figura retirada de Collins et al., *A review of Physics Simulators for Robotic Applications*, 2021

(ALGUNS) SIMULADORES MAIS FAMOSOS EM ROBÓTICA

Simulador	(Principais) Domínios de utilização	Motores de física	Formatos	Integrações
Gazebo	Manipuladores, Robôs móveis, manipulação destreza	ODE, Bullet, Simbody, DART	URDF, SDF, Collada	C++, ROS
Webots	Robótica móvel, desenvolvimento de algoritmos	ODE†ODE †	PROTO, URDF*	C,C++, Python, Java, Matlab, ROS
MuJoCo	Aprendizado por reforço, simulações com muitos contatos	MuJoCo	MJCF, OpenUSD, URDF*	C++, Python, ROS*
CoppeliaSim	Robótica industrial, manipuladores	ODE, Bullet, MuJoCo, Vortex, Newton	URDF, SDF, glTF, Formatos CAD	C/++, Python, Lua, Matlab*, ROS/2
Isaac Sim	Aprendizado por reforço, visão computacional	Newton	URDF, MJCF, OpenUSD, Formatos CAD	C++, Python, ROS/2
Drake	Pesquisa, controle ótimo, robôs subaquáticos terrestres	Customizada pelo usuário	YAML, URDF†, SDF†, MJCF†YAML †	Python, LCM, ROS 2*, Julia*

* Extensões externas, não são desenvolvida/mantidas pela mesma equipe do simulador

† Implementação específica, diferente do padrão

O SIMULADOR GAZEBO

Simulador atualmente desenvolvido e mantido pela Open Source Robotics Foundation (OSRF)

- Versão original (Gazebo Classic) desenvolvida até versão 11 (EOL) em 2025
- Fork atual se chama apenas Gazebo
 - Diferentes releases – atualmente está na release "Jetty"
 - Integração com ROS 2 Jazzy pela release "Harmonic"
- Sistema de bibliotecas disponível para integração e customização da simulação
- Descrição de mundos via arquivos .SDF
 - Modelos online podem ser carregados pelo sistema Fuel
- Sistema de plugins para carregar código externo (e.g. controlador de juntas,

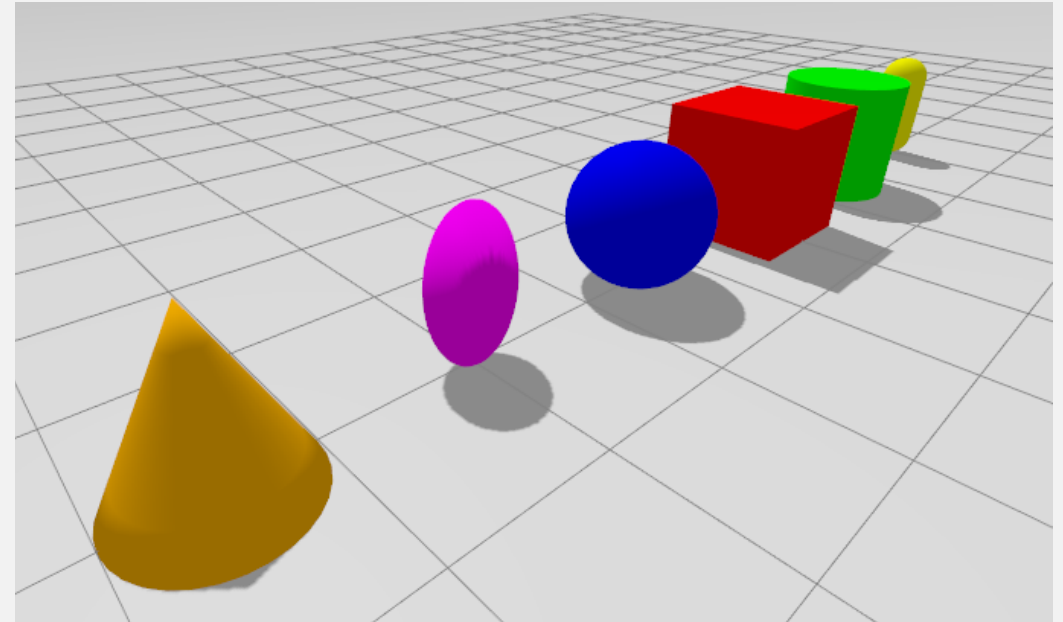


Mundo de armazém carregado dentro do Gazebo

SOBRE ARQUIVOS .SDF

SDF : Simulation Description Format ([Documentação](#))

- Formato de arquivo baseado em XML projetado para descrever ambientes e modelos em simulações
- Padrão do Gazebo
- Define tudo relacionado ao mundo (e até robôs)
 - Gravidade, Física, Iluminação, ...
- Suporta utilização e configuração de sensores
- É o irmão mais velho do URDF, conseguindo fazer tudo que ele e mais
 - ...incluindo cadeias cinemáticas fechadas!
 - Por baixo dos panos nossos URDF (dos robôs) são convertidos em SDF
 - Existem tags especiais que podemos incluir no URDF para usar funcionalidades específicas do Gazebo sem reescrever o robô inteiro

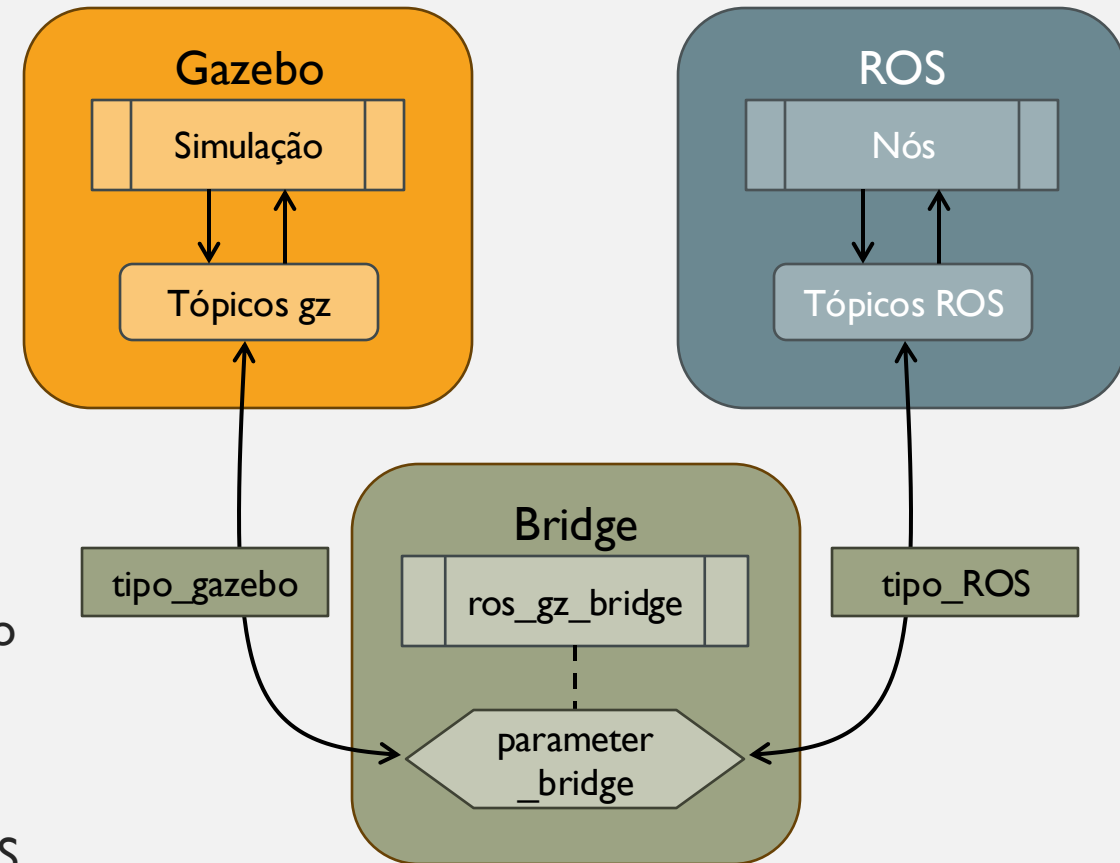


Mundo `shapes.sdf`, incluso nativamente no Gazebo. Os arquivos `.sdf` dos mundos padrão estão disponíveis por *default* no diretório `/usr/share/gz/gz-sim{versão}/worlds/`

INTEGRAÇÃO GAZEBO-ROS: ROS_GZ

Pacote de integração entre Gazebo e ROS

- Gazebo usa internamente um sistema de transporte similar ao do ROS
- Pacote permite comunicação entre mensagens no Gazebo Transport para ROS, e vice-versa
- [Disponível no github](#)
- Subpacotes:
 - `ros_gz_bridge`: Transporte bidirecional entre tópicos no Gazebo e tópicos no ROS
 - `ros_gz_image`: Imagens do Gazebo para ROS
 - `ros_gz_pointcloud`: Point clouds de Gazebo para ROS
 - `ros_gz_sim`: Utilitários para lançar Gazebo a partir do ROS



ROS_GZ_BRIDGE

Para abrir um canal de comunicação entre gazebo e ROS, podemos usar executável `parameter_bridge`, com a seguinte sintaxe:

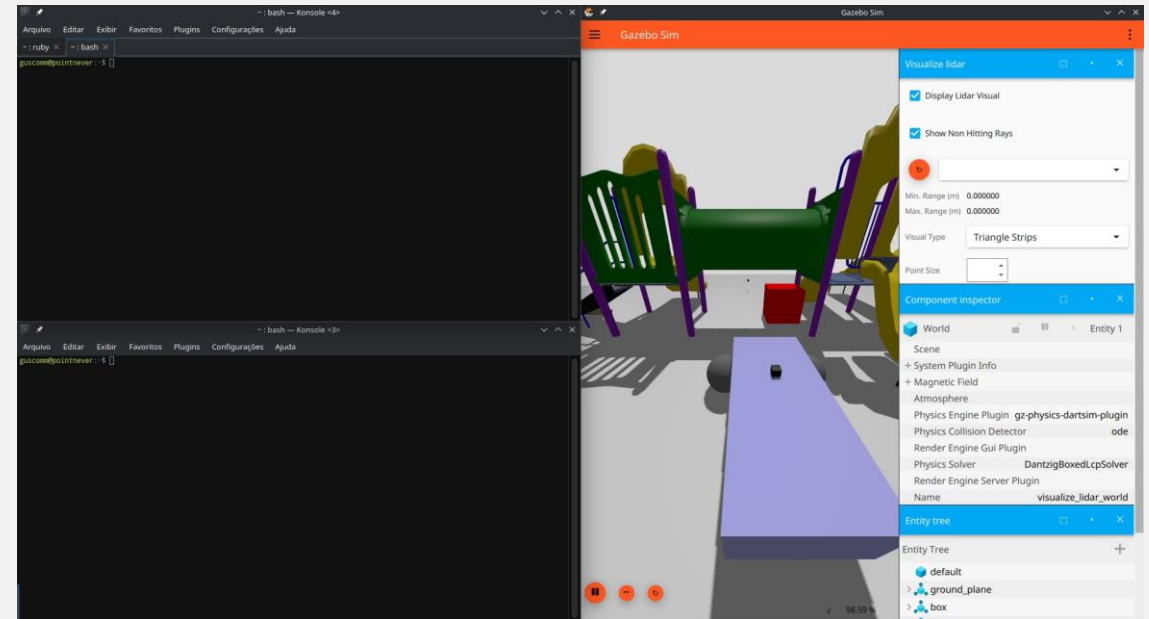
```
ros2 run ros_gz_bridge parameter_bridge  
/GAZEBO_TOPIC@ROS_MSG$GAZEBO_MSG
```

Onde:

- **/GAZEBO_TOPIC**: Nome do tópico dentro do Gazebo (ex: /Twist)
 - **ROS_MSG**: Tipo de mensagem equivalente no ROS (ex: geometry_msgs/msg/Twist)
 - **GAZEBO_MSG**: Tipo de mensagem usada internamente no gazebo (ex: gz.msgs.Twist)
- último símbolo (\$) dita o sentido de comunicação:
- **@**: Bidirecional , **[**: Gz → ROS e **]**: ROS → Gz

Exemplo: No vídeo rodamos o seguinte comando no terminal superior:

```
ros2 run ros_gz_bridge parameter_bridge  
/model/vehicle/cmd_vel@geometry_msgs/msg/Twist@ignition.msgs.Twist
```



Demonstração do `ros_gz_bridge` para enviar comandos do ROS no mundo `visualize_lidar.sdf`

Uma lista de equivalências entre tipos de mensagens do ROS e Gazebo pode ser encontrada na documentação:
https://docs.ros.org/en/jazzy/p/ros_gz_bridge/

INTEGRAÇÃO GAZEBO-ROS: ROS_GZ

- Nem todas os tipos de mensagem no ROS tem seu equivalente no Gazebo
- Pode se configurar o bridge através de arquivos de launch também
 - A bridge também pode ser definida em um YAML
- Para controle mais sofisticado (PID, controladores de juntas via `ros2_control`), apenas `ros_gz` não basta: é preciso configurar controladores dentro do Gazebo (via `gz_ros2_control`).

```
<launch>
  <arg name="world_sdf" default="empty.sdf"/>
  <arg name="bridge_name" default="ros_gz_bridge"/>
  <arg name="config_file" default=""/>

  <!-- (Gazebo) -->
  <gz_server
    world_sdf_file="$(var world_sdf)"
    container_name="ros_gz_container"
    create_own_container="True"
    use_composition="True" />

  <!-- Bridge ROS Gazebo -->
  <ros_gz_bridge
    bridge_name="$(var bridge_name)"
    config_file="$(var config_file)"
    container_name="ros_gz_container"
    use_composition="True"
    use_respawn="False"
    log_level="info" >

    <!-- Topico ex: Twist bidirecional -->
    <topic ros_topic_name="/model/robot/cmd_vel"
      gz_topic_name="/model/robot/cmd_vel"
      ros_type_name="geometry_msgs/msg/Twist"
      gz_type_name="gz.msgs.Twist"
      direction="BIDIRECTIONAL"
      lazy="False"
      qos_profile="default" />

  </ros_gz_bridge>
</launch>
```

ROS_GZ_SIM

- Fornece ferramentas para rodar/controlar o **Gazebo** a partir do ROS 2
 - Com nossos comandos padrão que já conhecemos (launch, run, ...)
- launch files prontos para simular mundos SDF
 - Ex: `ros2 launch ros_gz_sim gz_sim.launch.py gz_args:="shapes.sdf"`

ROS_GZ_SIM_DEMOS

- Um pacote de demonstrações.
- Exemplos prontos para usar a ponte (bridge), simular robôs, sensores e publicação de dados
 - Robô diff-drive (direcional), sensores de pressão, câmeras, ...
 - Ex: `ros2 launch ros_gz_sim_demos diff_drive.launch.py`
 - Utiliza do `ros_gz_sim`

ROS_GZ_IMAGE

- Para imagens: cria uma ponte unidirecional da Gazebo Transport (`gz::msgs::Image`) para ROS 2 usando `image_transport`
 - Usamos esse pacote para ter imagens no "ambiente" ros, mas que são geradas no gazebo
- launch files prontos para simular mundos SDF
 - Ex: `ros2 launch ros_gz_sim_demos image_bridge.launch.py`
 - Ex: `ros2 launch ros_gz_sim_demos camera.launch.py`

ROS_GZ_POINT_CLOUD

- Fornece plugins para publicar point clouds do Gazebo para o ROS 2.
- Para simular sensores como LIDARs no Gazebo e enviar os dados para o ROS para uso em algoritmos de mapeamento, percepção

INTEGRAÇÃO GAZEBO- ROS2_CONTROL: GZ_ROS2_CONTROL

Pacote gz_ros2_control

- Fornece um plugin de sistema para o Gazebo-Sim, que instancia um Controller Manager e conecta ele a um modelo Gazebo
- Para usá-lo, duas tags devem ser adicionadas e configuradas no arquivo URDF/SDF do robô:
 - Uma tag `<plugin>` com o item `gz_ros2_control/GazeboSimSystem` na tag `<hardware>` de `<ros2_control>` (que será lida pelo `ros2_control`)
 - Uma tag `<gazebo>` com o plugin `gz_ros2_control-system` (que será lida pelo Gazebo), incluindo um arquivo de parâmetros de configuração

```
<robot>
  ... Resto da descrição do robô, antes ou depois das tags abaixo→

  <ros2_control name="sistema_motor" type="system"
    <hardware> HardwareInterface para conexão com o Gazebo→
      <plugin>gz_ros2_control/GazeboSimSystem</plugin>
    </hardware>

    Junta atuada pelo motor→
    <joint name="nome_da_junta_no_urdf">
      Interfaces de comando e estados expostas;
      aqui só controlamos posição →
      <command_interface name="position" />
      <state_interface name="position" />
      <state_interface name="velocity" />
      <state_interface name="effort" />
    </joint>
  </ros2_control>

  <gazebo>
    <plugin filename="gz_ros2_control-system"
      name="gz_ros2_control::GazeboSimROS2ControlPlugin">
      <parameters>$(find motor_bringup)/config/params.yaml</parameters>
    </plugin>
  </gazebo>
</robot>
```

Parte de arquivo urdf, com tags para integração do
ros2_control ao Gazebo

DEMONSTRAÇÃO GAZEBO-ROS2_CONTROL

Pacote `gz_ros2_control_demos` ([github](#))

- Instalação via linha de comando (ou download do github):

```
sudo apt install ros-$ROS_DISTRO-gz-ros2-control-demos
```

Vamos rodar a demonstração `cart_pole`:

- Em um terminal, chame o arquivo de launch:

```
ros2 launch gz_ros2_control_demos cart_example_effort.launch.py
```

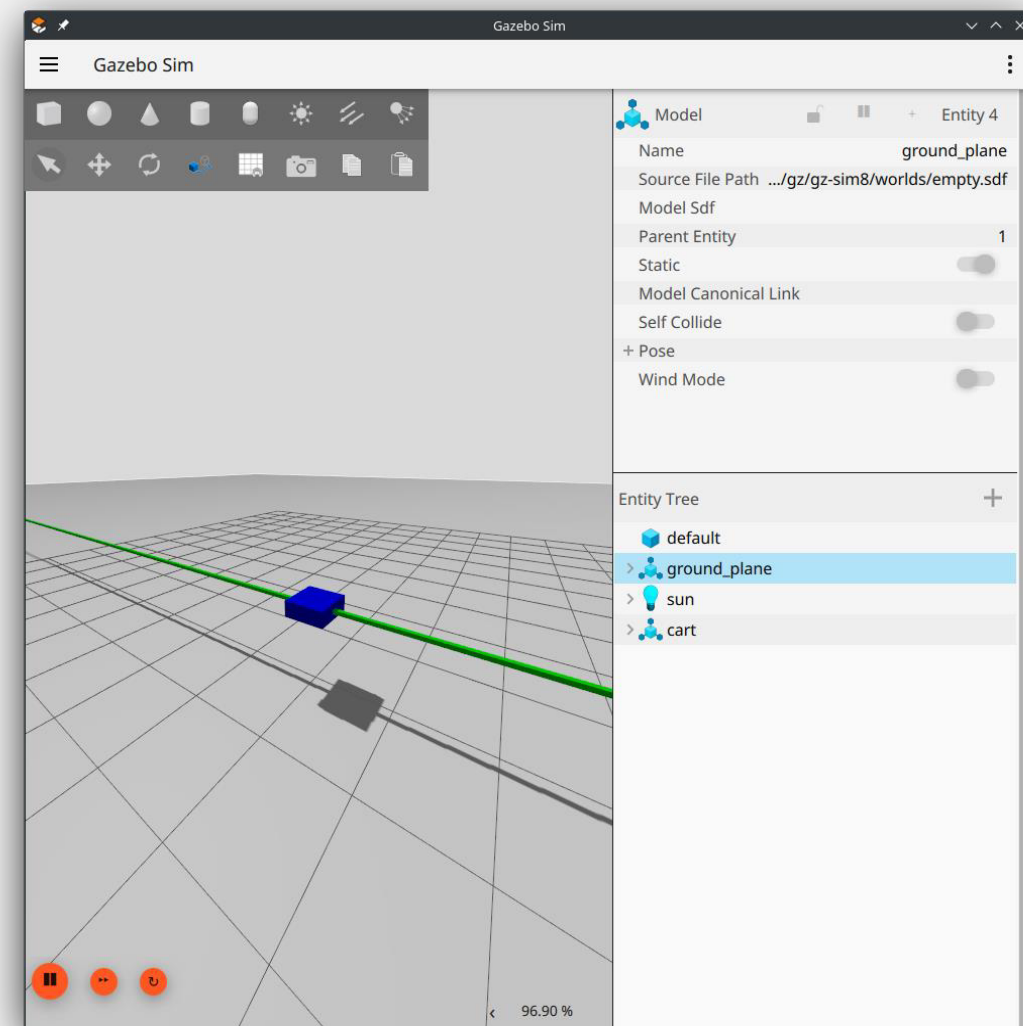
- Em outro terminal, verifique os controladores carregados e as interfaces disponíveis::

```
ros2 control list_controllers
```

```
ros2 control list_hardware_interfaces
```

- Plote a posição, velocidade e esforço do bloco com `rqt_plot`
- Chame o controlador de esforço incluso no mesmo exem

```
ros2 run gz_ros2_control_demos example_effort
```



Vista do exemplo `cart_example_effort` do pacote `gz_ros2_control_demos`

DEMONSTRAÇÃO GAZEBO-ROS2_CONTROL

Dois exercícios mais simples:

- Altere os ganhos P, I, D, do controlador `joint_trajectory_controller`; veja se você consegue um desempenho sem *overshoot* da velocidade.
- Compare os sinais vindo quando usamos um controlador via interface de esforço, comparado aos controles via interface de posição e de velocidade. Por que eles são tão diferentes?

```
ros2 launch gz_ros2_control_demos cart_example_position.launch.py &  
ros2 run gz_ros2_control_demos example_position
```

```
ros2 launch gz_ros2_control_demos cart_example_velocity.launch.py &  
ros2 run gz_ros2_control_demos example_velocity
```

E um exercício (consideravelmente) mais complicado:

- Faça um pacote para implementar um controlador de posição que aplique uma ação de controle de esforço, proporcional ao erro de posição do carrinho com relação à referência de posição nula. (Será resolvido para a próxima aula)

Gráfico da posição, esforço e velocidade do bloco, gerado no plotjuggler

